

DATABASE DESIGN DOCUMENT

AI-Smart Campus System

Database design for authentication, academic records, smart campus workflows, recommendations, tasks, notices, and risk alerts.

Field	Details
Course	CSE 4204 - Mobile Computing Lab
Team	CSE4204-8A-T07
Technology Stack	Laravel, MySQL, Sanctum API Authentication
Prepared Date	June 29, 2026
Document Type	Functional Database Design Document

Team Members

Role	Name	Student ID
Team Leader	Md. Nazmus Shakib	11220320852
AI	Samira Akter Mitu	11220320858
Frontend	Tanvin Sadik Dhrubo	11220320860
Backend	Khan Waziur Rahman	11220320861

1. Database Overview

The AI-Smart Campus System database is designed to support a role-based academic platform where students, faculty, and administrators can authenticate, manage campus tasks, publish learning resources, monitor academic performance, and receive intelligent recommendations.

The design uses a relational MySQL structure with primary keys, foreign keys, timestamps, and normalized module-based tables. Laravel Sanctum is used for secure API token authentication.

1.1 Core Modules

- Authentication and user management
- Student, faculty, admin, and course data
- Assessment and attendance tracking
- Performance analysis and risk alerts
- Learning resources and campus tasks
- AI/course recommendations and notices

1.2 Design Goals

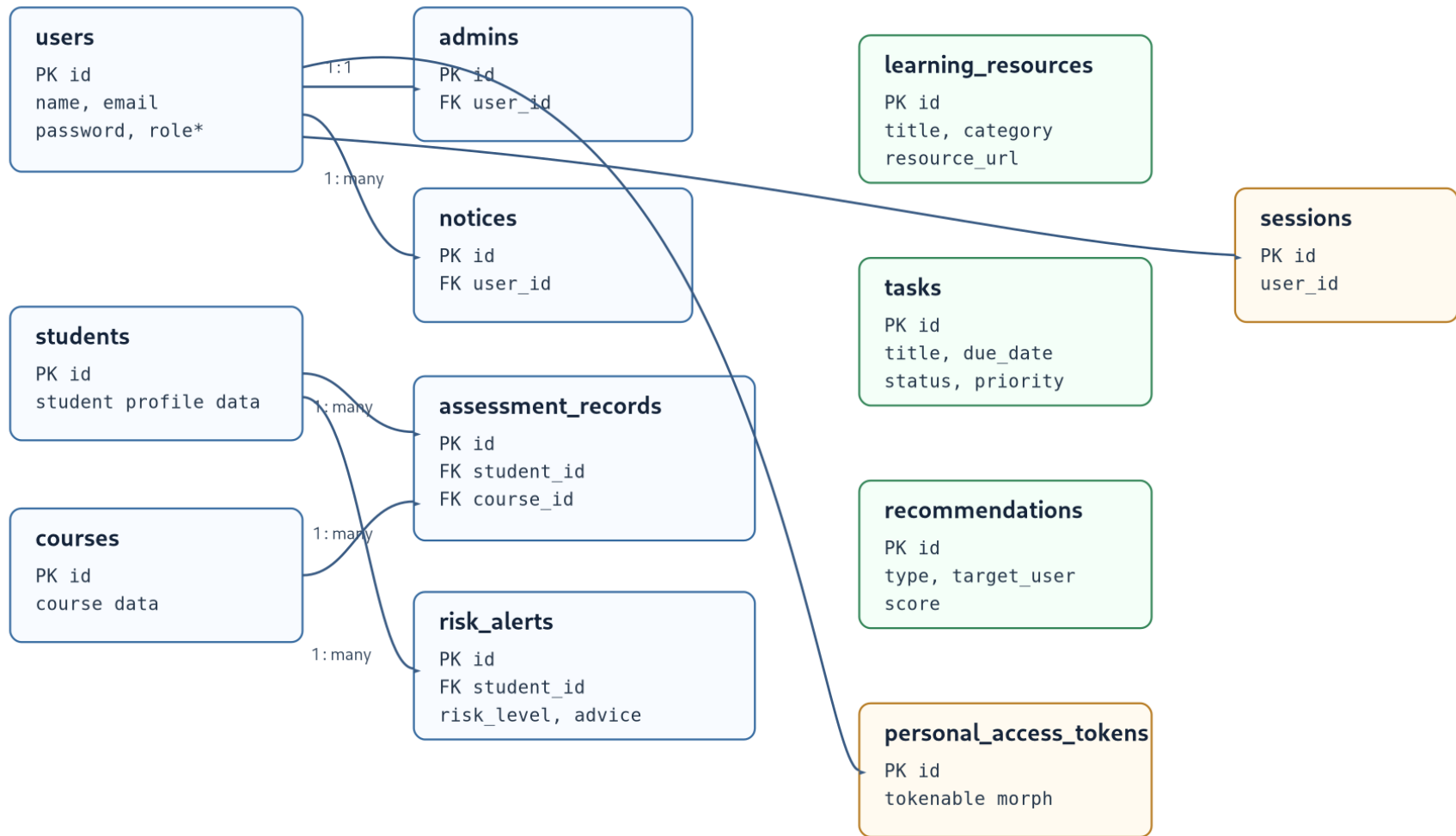
- Keep user login data separate from academic records.
- Store assessment, attendance, and risk data in independent tables.
- Use foreign keys where direct entity relationships exist.
- Keep API module tables simple and easy to extend.
- Maintain a structure that can be implemented directly in Laravel migrations.

1.3 Implementation Scope

The current backend includes project-level migrations and API module migrations. This document therefore includes both the conceptual project tables and the implemented API tables used by the Laravel backend. API module tables can later be linked to users, students, and courses as the system expands.

2. Entity Relationship Diagram

High-level ERD showing the main project relationships and implemented Laravel API module tables.



Green boxes: API module tables. Orange boxes: Laravel/Sanctum support tables.

* role and approval status are used by the API user model/controller.

3. Relationship Summary

Relationship	Cardinality	Purpose
users.id to admins.user_id	One user can map to one admin profile.	Connects login identity with admin profile data.
users.id to notices.user_id	One user can publish many notices.	Tracks the author/publisher of campus notices.
students.id to assessment_records.student_id	One student can have many assessment records.	Stores quiz, assignment, and midterm marks per student.
courses.id to assessment_records.course_id	One course can have many assessment records.	Links assessment marks to a specific course.
students.id to risk_alerts.student_id	One student can have many risk alerts.	Stores predicted academic risk and intervention advice.
users to personal_access_tokens	One user can have many API tokens.	Supports Laravel Sanctum authentication.

4. Data Dictionary

4.1 users

Column	Type	Constraint	Description
id	bigint unsigned	Primary key, auto increment	Unique user identifier.
name	varchar	Required	User full name.
email	varchar	Required, unique	User email address used for login.
email_verified_at	timestamp	Nullable	Email verification timestamp.
password	varchar	Required	Hashed password.
remember_token	varchar	Nullable	Laravel remember token.
role	enum/string	Application field	Role values: student, faculty, admin.
approval_status	enum/string	Application field	Account approval state used by the login flow.
created_at, updated_at	timestamp	Nullable	Record audit timestamps.

4.2 admins

Column	Type	Constraint	Description
id	bigint unsigned	Primary key	Unique admin profile identifier.
user_id	bigint unsigned	Foreign key to users, cascade delete	Linked login account.
created_at, updated_at	timestamp	Nullable	Record audit timestamps.

4.3 students

Column	Type	Constraint	Description
id	bigint unsigned	Primary key	Unique student identifier.
created_at, updated_at	timestamp	Nullable	Record audit timestamps.

4.4 faculty

Column	Type	Constraint	Description
id	bigint unsigned	Primary key	Unique faculty identifier.
created_at, updated_at	timestamp	Nullable	Record audit timestamps.

4.5 courses

Column	Type	Constraint	Description
id	bigint unsigned	Primary key	Unique course identifier.
created_at, updated_at	timestamp	Nullable	Record audit timestamps.

4.6 assessment_records

Column	Type	Constraint	Description
id	bigint unsigned	Primary key	Unique assessment record identifier.
student_id	bigint unsigned	Foreign key to students, cascade delete	Student who owns the marks.
course_id	bigint unsigned	Foreign key to courses, cascade delete	Course for the assessment.
quiz_marks	float	Default 0	Quiz mark total.
assignment_marks	float	Default 0	Assignment mark total.
mid_marks	float	Default 0	Midterm mark total.
created_at, updated_at	timestamp	Nullable	Record audit timestamps.

4.7 notices

Column	Type	Constraint	Description
id	bigint unsigned	Primary key	Unique notice identifier.
user_id	bigint unsigned	Nullable foreign key to users, null on delete	User who published the notice.
title	varchar(255)	Required	Notice headline.
description	text	Required	Notice details.
publish_date	timestamp	Nullable	Date/time the notice is published.
created_at, updated_at	timestamp	Nullable	Record audit timestamps.

4.8 risk_alerts

Column	Type	Constraint	Description
id	bigint unsigned	Primary key	Unique risk alert identifier.
student_id	bigint unsigned	Foreign key to students, cascade delete	Student at risk.
risk_level	enum	low, medium, high; default low	Predicted academic risk level.
prediction	varchar(255)	Required	Prediction label or summary.
advice	text	Nullable	Suggested intervention or guidance.
created_at, updated_at	timestamp	Nullable	Record audit timestamps.

4.9 learning_resources

Column	Type	Constraint	Description
id	bigint unsigned	Primary key	Unique learning resource identifier.
title	varchar	Required	Resource title.
description	text	Nullable	Resource summary or details.
category	varchar	Required	Subject or category, for example CSE.
resource_type	varchar	Required	Resource format such as PDF, video, link, note.
resource_url	varchar	Nullable	External or uploaded resource URL.
uploaded_by	varchar	Nullable	Name or identifier of uploader.
created_at, updated_at	timestamp	Nullable	Record audit timestamps.

4.10 tasks

Column	Type	Constraint	Description
id	bigint unsigned	Primary key	Unique campus task identifier.
title	varchar	Required	Task title.
description	text	Nullable	Task details.
assigned_to	varchar	Nullable	Student/faculty/admin assigned to the task.
due_date	date	Nullable	Task deadline.
status	enum	pending, in_progress, completed; default pending	Task workflow state.
priority	enum	low, medium, high; nullable	Task priority level.

Column	Type	Constraint	Description
created_at, updated_at	timestamp	Nullable	Record audit timestamps.

4.11 recommendations

Column	Type	Constraint	Description
id	bigint unsigned	Primary key	Unique recommendation identifier.
title	varchar	Required	Recommendation title.
description	text	Nullable	Recommendation details.
recommendation_type	varchar	Required	Type such as study_plan, course_advice, risk_support.
target_user	varchar	Nullable	User or student receiving the recommendation.
score	float	Nullable	Confidence score, priority score, or performance score.
created_at, updated_at	timestamp	Nullable	Record audit timestamps.

4.12 Supporting Project Tables

Table	Main Columns	Purpose
academic_records	id, timestamps	Reserved for semester, CGPA, and academic history records.
attendance_records	id, timestamps	Reserved for attendance tracking by student/course/date.
performance_metrics	id, timestamps	Reserved for analytics values used in dashboards and risk prediction.
course_recommendations	id, timestamps	Reserved for AI-generated future course advice.

4.13 Laravel System Tables

Table	Important Columns	Purpose
personal_access_tokens	tokenable_type, tokenable_id, token, abilities, expires_at	Stores Sanctum API tokens.
password_reset_tokens	email, token, created_at	Supports password reset workflow.
sessions	id, user_id, ip_address, user_agent, payload, last_activity	Stores session data when database sessions are enabled.
cache, cache_locks	key, value, expiration	Stores cache and cache lock data.
jobs, job_batches, failed_jobs	Queue payload, status, and failure fields	Supports Laravel queue processing.

5. Normalization Notes

- Each table has a single primary key and stores data for one main entity or workflow.
- Authentication data is kept in users, while academic and workflow data is stored in separate module tables.
- Assessment marks use separate foreign keys for students and courses, avoiding duplicated student/course data.
- Risk alerts are separated from assessment records so multiple predictions can exist for one student over time.
- API module tables are independent at the current implementation stage and can later be connected to users, students, or courses through foreign keys.

6. API Table Mapping

API Resource	Database Table	Supported Actions
Authentication	users, personal_access_tokens	Register, Login, Profile, Logout
Learning Resources	learning_resources	Create, List, Show, Update, Delete
Campus Tasks	tasks	Create, List, Show, Update, Delete
Recommendations	recommendations	Create, List, Show, Update, Delete

7. Implementation Checklist

Requirement	Status in Design	Notes
Primary Keys	Included	Every table uses an id primary key.
Foreign Keys	Included where direct relationships exist	Implemented for users/admins/notices, students/assessment, courses/assessment, and students/risk alerts.
Relationships	Documented	Relationship summary identifies cardinality and purpose.
Validations	Defined at application level	Required, unique, nullable, enum, and default rules are described in the data dictionary.
Scalability	Supported	Module-based tables can be expanded without redesigning the whole schema.

8. Conclusion

This database design provides the relational foundation for the AI-Smart Campus System. It supports authentication, academic monitoring, resource management, campus tasks, notices, AI recommendations, and risk alert workflows. The schema is functional for the current Laravel backend and can be expanded with additional foreign keys and profile fields as project modules become more detailed.